



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Introducción A La Programación Y Computación 2

[Sección X]

Escuela de Ingeniería de Ciencias Y Sistemas

NOMBRE DEL TUTOR

UNIDAD 4:

Estructura de Datos

Anuncios Importantes



Anuncio 1



Anuncio 2



Anuncio 3



Anuncio 4



Anuncio 5

Agenda



Introducción a estructura de datos



Listas nativas en C#



Ficheros



Diccionarios, Tuplas



Expresiones regulares

COMPETENCIA(S) QUE DESARROLLAREMOS



Crea estructuras de datos dinámicas (listas enlazadas, doblemente enlazadas y circulares) mediante POO en C# e integrado datos desde archivos XML para implementar sistemas de gestión de datos eficientes que manejen relaciones complejas.



VALOR DE LA SEMANA

INTEGRIDAD

La integridad se aplica cuando los ingenieros actúan con honestidad, asegurando que sus sistemas sean seguros y confiables. Evitan comprometer la calidad por presiones externas o atajos.

"La integridad es hacer lo correcto, incluso cuando nadie está mirando." – C.S. Lewis

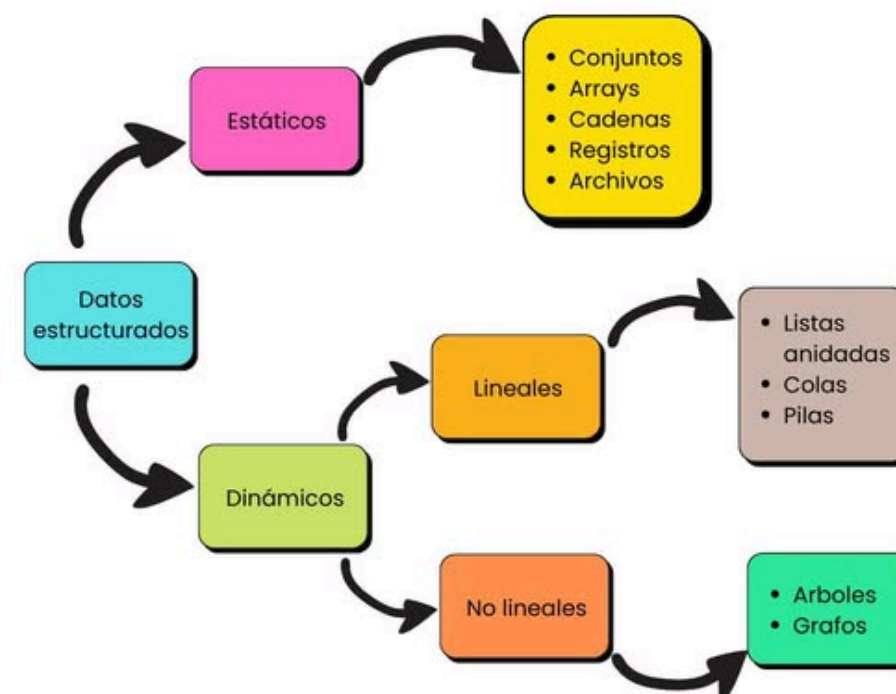
INTRODUCCIÓN DE ESTRUCTURAS DE DATOS

ESTÁTICAS

Son aquellas en las que el tamaño ocupado en memoria se define antes de que el programa se ejecute y no puede modificarse dicho tamaño durante la ejecución del programa.

DINÁMICAS

Son aquellas en la que el tamaño ocupado en memoria puede modificarse durante la ejecución del programa.



ARRAY

Un array es una colección de elementos que se almacenan en posiciones contiguas de memoria. En C#, como en muchos otros lenguajes de programación, los arrays son de tamaño fijo y cada elemento del array se accede mediante un índice numérico.

```
// Declaración e inicialización  
int[] numeros = new int[5] { 10, 20, 30, 40, 50 };  
  
// Acceso a elementos  
int primerNumero = numeros[0]; // 10  
int tercerNumero = numeros[2]; // 30
```


ARRAYS MULTIDIMENSIONALES

Los arrays multidimensionales son estructuras de datos que permiten organizar información en múltiples dimensiones, como filas y columnas. Se utilizan cuando se necesita representar datos de naturaleza tabular o matricial, como tableros de juego, tablas o cuadrículas. Existen dos tipos principales: los arrays rectangulares, donde todas las filas tienen el mismo número de columnas formando una estructura uniforme, y los arrays escalonados, que son arrays de arrays donde cada fila puede tener un número diferente de elementos, permitiendo estructuras más flexibles.

```
// Declaración e inicialización - Tabla de 3 filas x 2 columnas
int[,] tabla = new int[3, 2]
{
    { 1, 2 },
    { 3, 4 },
    { 5, 6 }
};

// Acceso a elementos
int elemento = tabla[0, 1]; // 2 (fila 0, columna 1)
int otro = tabla[2, 0];     // 5 (fila 2, columna 0)
```

LISTAS NATIVAS DE C#

Una lista es una estructura de datos en C# utilizada para contener listas de objetos del mismo tipo. Son genéricas, lo que significa que el tipo en la lista se declara en la inicialización. Las listas son específicamente una estructura de datos ordenada, lo que significa que el orden en que agregas elementos a la lista es el orden de los elementos en esa lista, incluyendo valores duplicados.

```
// Declaración e inicialización
List<string> nombres = new List<string> { "Ana", "Luis", "María" };

// Agregar elementos
nombres.Add("Carlos");

// Acceso a elementos
string primerNombre = nombres[0]; // "Ana"
string ultimoNombre = nombres[3]; // "Carlos"

// Eliminar elementos
nombres.Remove("Luis");

// Verificar cantidad de elementos
int cantidad = nombres.Count; // 3
```

TIPOS DE LISTAS EN C#

1. **List<T>** - Lista genérica estándar. Es dinámica y permite agregar, insertar, eliminar y buscar elementos de forma eficiente.
2. **ReadOnlyList<T>** - Lista de solo lectura que no permite agregar o eliminar elementos después de la inicialización. Se usa cuando quieres pasar datos sin permitir modificaciones.
3. **LinkedList<T>** - Lista enlazada que permite agregar elementos relativos a una posición específica (antes/después de un nodo). Mejor cuando necesitas inserciones/eliminaciones frecuentes en posiciones específicas.
4. **ImmutableList<T>** - Lista inmutable que retorna una nueva versión cada vez que se agregan o eliminan elementos. Thread-safe, pero con impacto en rendimiento.

TIPOS DE LISTAS EN C#

- 5. **SortedList<K,V>** - Lista ordenada que maneja pares clave-valor y los ordena automáticamente por clave.
- 6. **Stack<T>** - Pila (LIFO - Last In, First Out). Útil para operaciones donde el último elemento agregado debe procesarse primero.
- 7. **Queue<T>** - Cola (FIFO - First In, First Out). Útil cuando necesitas procesar elementos en el orden que llegaron.
- 8. **Array** - Arreglo de tamaño fijo. Más antiguo y menos flexible que List<T>, pero fundamental en C#.

DICCIONARIO

Un Diccionario en C# es una colección genérica que permite almacenar pares clave-valor. Proporcionan una forma muy eficiente de realizar búsquedas, inserciones y eliminaciones.

Un diccionario en C# se representa mediante `Dictionary<TKey, TValue>` e internamente utiliza la estructura `KeyValuePair<TKey, TValue>` para almacenar cada elemento. La característica principal es que las claves deben ser únicas: no pueden existir dos entradas con la misma clave en el diccionario.

```
// Declaración e inicialización
Dictionary<string, int> edades = new Dictionary<string, int>
{
    { "Ana", 25 },
    { "Luis", 30 },
    { "María", 28 }
};

// Agregar elementos
edades.Add("Carlos", 35);

// Acceso a elementos por clave
int edadAna = edades["Ana"]; // 25

// Actualizar elementos
edades["Luis"] = 31;
```


OPERACIONES CON DICCIONARIOS EN C#

1. Crear y Añadir Elementos - Los diccionarios se pueden inicializar con valores al momento de su creación o agregar elementos posteriormente mediante el método `Add`, especificando la clave y el valor correspondiente.

2. Acceder a Elementos - Para acceder a los elementos existen tres formas: mediante el indexer usando la clave, por posición usando `ElementAt` (que devuelve un `KeyValuePair`), o mediante `TryGetValue` que es la forma más óptima ya que evita excepciones si la clave no existe, devolviendo un booleano que indica si se encontró el elemento.

3. Actualizar Elementos - Para modificar un elemento, primero se debe verificar que la clave exista usando `ContainsKey`, y luego actualizar el valor mediante el indexer con la clave correspondiente.

OPERACIONES CON DICCIONARIOS EN C#

4. Eliminar Elementos - Los elementos se pueden eliminar individualmente mediante el método Remove especificando la clave, o eliminar todos los elementos del diccionario mediante el método Clear.

5. Verificar Existencia - El método ContainsKey permite verificar si una clave específica existe en el diccionario, devolviendo verdadero o falso según corresponda.

Nota importante: La clave (Key) debe ser única en el diccionario. Intentar agregar una clave duplicada generará una excepción.

TUPLAS

En C#, las tuplas son estructuras de datos ligeras que permiten agrupar múltiples valores de diferentes tipos en una sola unidad.

A diferencia de las clases o structs, las tuplas son más sencillas y están diseñadas para simplificar el manejo de múltiples valores (nos evitan tener que definir tipos adicionales).

En C# las tuplas son inmutables. Eso significa que una vez creadas, sus valores no pueden ser cambiados.

C# Tuple

```
(string ItemName, double Price) itemPrice = ("Socks", 12.99);
```

```
Console.WriteLine(itemPrice.ItemName); // Socks  
Console.WriteLine(itemPrice.Price); // 12.99
```

```
(string name, _) = itemPrice;  
Console.WriteLine(name); // Socks
```



OPERACIONES CON TUPLAS EN C#

- 1. Crear Tuplas** - Se pueden crear de dos formas: especificando los tipos genéricos en el constructor o utilizando el método estático `Create` que infiere automáticamente los tipos. Las tuplas pueden contener hasta 8 elementos de diferentes tipos.
- 2. Acceder a Elementos** - Se accede a los elementos mediante las propiedades `Item1`, `Item2`, `Item3`, etc., según la posición del elemento en la tupla. En el caso de `ValueTuple`, también se pueden asignar nombres personalizados a cada elemento para mejorar la legibilidad del código.
- 3. Pasar como Parámetros** - Las tuplas pueden pasarse como parámetros a métodos, permitiendo enviar múltiples valores agrupados en un solo argumento. Esto simplifica las firmas de métodos que necesitan recibir varios datos relacionados.

OPERACIONES CON TUPLAS EN C#

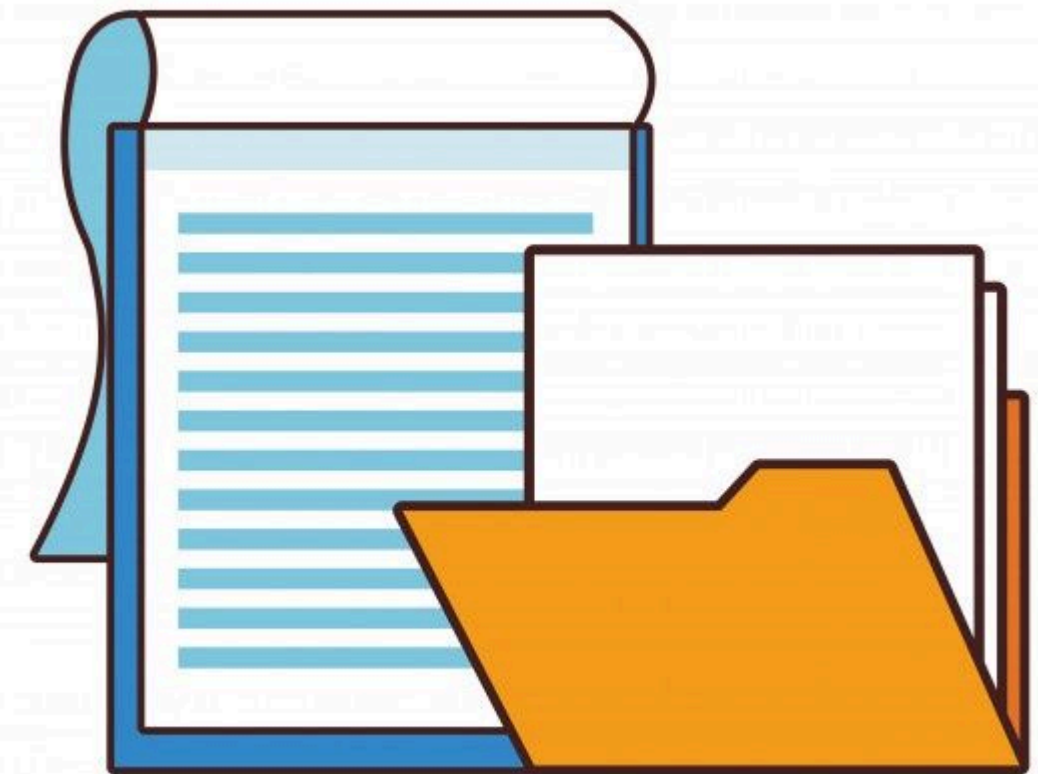
4. Devolver desde Métodos - El uso más común de las tuplas es como tipo de retorno, permitiendo que un método devuelva múltiples valores sin necesidad de crear clases auxiliares o usar parámetros ref u out.

5. Deconstruir ValueTuple - Los ValueTuple pueden deconstruirse directamente en variables individuales, asignando cada elemento de la tupla a una variable separada en una sola operación. Se puede usar el descarte (_) para ignorar elementos que no se necesiten.

Diferencia clave: Tuple es un tipo de referencia (reference type) ubicado en el heap, mientras que ValueTuple es un tipo de valor (value type) con mejor rendimiento y sintaxis más simple.

FICHEROS

Un fichero o archivo es una estructura de datos abstracta que representa una colección organizada de información almacenada de manera persistente en un dispositivo de almacenamiento secundario no volátil. Los datos contenidos en un fichero mantienen una relación lógica entre sí y permanecen almacenados independientemente del ciclo de vida de los procesos o aplicaciones que los manipulan.



FICHERO

La característica fundamental de persistencia implica que la información se conserva en el medio de almacenamiento (discos magnéticos, unidades de estado sólido, medios ópticos, etc.) más allá de la finalización del programa que los creó e incluso tras el apagado del sistema. Esta persistencia solo se ve comprometida por fallos críticos del hardware, corrupción de datos o eliminación intencional.



APERTURA DE FICHEROS

La lectura y escritura de ficheros son operaciones fundamentales de entrada/salida (I/O) que permiten a las aplicaciones interactuar con el sistema de almacenamiento persistente. La lectura se realiza típicamente mediante la clase `StreamReader`, que abre el archivo, lee su contenido línea por línea mediante el método `ReadLine`, y cierra el archivo una vez finalizada la operación. La escritura se implementa mediante la clase `StreamWriter`, que permite crear o abrir archivos, escribir datos usando métodos como `WriteLine` o `Write`, y especificar parámetros como el modo de apertura (sobrescritura o anexo) y la codificación del texto (ASCII, Unicode, UTF-8, etc.).

PASOS PARA LEER UN ARCHIVO DE TEXTO EN C#

- **Importar el espacio de nombres necesario** - Incluir System.IO para acceder a las clases de entrada/salida.
- **Crear una instancia de StreamReader** - Pasar la ruta completa del archivo como parámetro al constructor.
- **Leer el contenido del archivo** - Utilizar el método ReadLine para leer línea por línea hasta llegar al final del archivo (cuando devuelve null).
- **Procesar los datos leídos** - Realizar las operaciones necesarias con cada línea leída (mostrar, almacenar, procesar).
- **Cerrar el archivo** - Invocar el método Close para liberar los recursos y desbloquear el archivo.
- **Implementar manejo de excepciones** - Envolver el código en bloques try-catch-finally para manejar posibles errores como archivos inexistentes o inaccesibles.

PASOS PARA ESCRIBIR UN ARCHIVO DE TEXTO EN C#

- **Importar el espacio de nombres necesario** - Incluir `System.IO` y opcionalmente `System.Text` para especificar codificación.
- **Crear una instancia de `StreamWriter`** - Pasar la ruta del archivo, el modo de apertura (sobrescribir o anexar) y opcionalmente la codificación deseada.
- **Escribir datos en el archivo** - Utilizar `WriteLine` para escribir líneas completas con salto de línea, o `Write` para escribir sin salto automático.
- **Cerrar el archivo** - Invocar el método `Close` para asegurar que todos los datos se escriban y liberar los recursos del sistema.
- **Implementar manejo de excepciones** - Usar bloques `try-catch-finally` para garantizar que el archivo se cierre correctamente incluso si ocurren errores durante la escritura.

EXPRESIONES REGULARES

Una expresión regular es un modelo con el que el motor de expresiones regulares intenta buscar una coincidencia en el texto de entrada. Un modelo consta de uno o más literales de carácter, operadores o estructuras. Las expresiones regulares son una herramienta fundamental para el procesamiento y validación de texto, permitiendo definir patrones de búsqueda complejos mediante una sintaxis especializada. Estos patrones pueden utilizarse para validar formatos (como correos electrónicos o números de teléfono), extraer información específica de textos, reemplazar contenido o dividir cadenas según criterios determinados. El motor de expresiones regulares interpreta estos patrones y los aplica sobre cadenas de texto, evaluando si existe coincidencia total o parcial según las reglas definidas, lo que convierte a las expresiones regulares en una poderosa herramienta para manipulación y análisis de datos textuales en programación.

EXPRESIONES REGULARES

Este ejemplo utiliza expresiones regulares para validar direcciones de correo electrónico. Se define un patrón que especifica las reglas que debe cumplir un email válido: debe comenzar con caracteres alfanuméricos, guiones o puntos, seguido del símbolo @, luego el dominio que también contiene caracteres alfanuméricos y puntos, y finalmente una extensión de al menos dos letras. El método `Regex.IsMatch()` compara cada cadena de texto contra el patrón definido y devuelve verdadero si la cadena cumple con todas las reglas especificadas en la expresión regular, o falso en caso contrario. En el ejemplo, el primer correo cumple con el patrón y se valida como correcto, mientras que el segundo no contiene el símbolo @ ni un dominio válido, por lo que se marca como inválido.

```
// Patrón para validar un correo electrónico simple
string patron = @"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$";

string email1 = "usuario@ejemplo.com";
string email2 = "correo_invalido";

// Validar correos
bool esValido1 = Regex.IsMatch(email1, patron);
bool esValido2 = Regex.IsMatch(email2, patron);

Console.WriteLine($"{email1} es válido: {esValido1}"); // True
Console.WriteLine($"{email2} es válido: {esValido2}"); // False
```


CONCEPTOS CLAVE APRENDIDOS

- 1. TDA (Tipo de Dato Abstracto)** - Modelo que define operaciones sobre datos sin especificar su implementación. Se enfoca en el "qué" hacer, no en el "cómo" hacerlo.
- 2. Estructuras de Datos Nativas en C#** - C# proporciona Arrays (tamaño fijo), List<T> (dinámicas), Dictionary<TKey, TValue> (clave-valor), y Tuplas (agrupación de tipos diferentes).
- 3. Persistencia de Ficheros** - Los ficheros almacenan datos permanentemente en dispositivos no volátiles. Se utilizan StreamReader para leer y StreamWriter para escribir.
- 4. Expresiones Regulares** - Patrones de texto que permiten buscar, validar y manipular cadenas mediante sintaxis especializada, útiles para validaciones y búsquedas complejas.

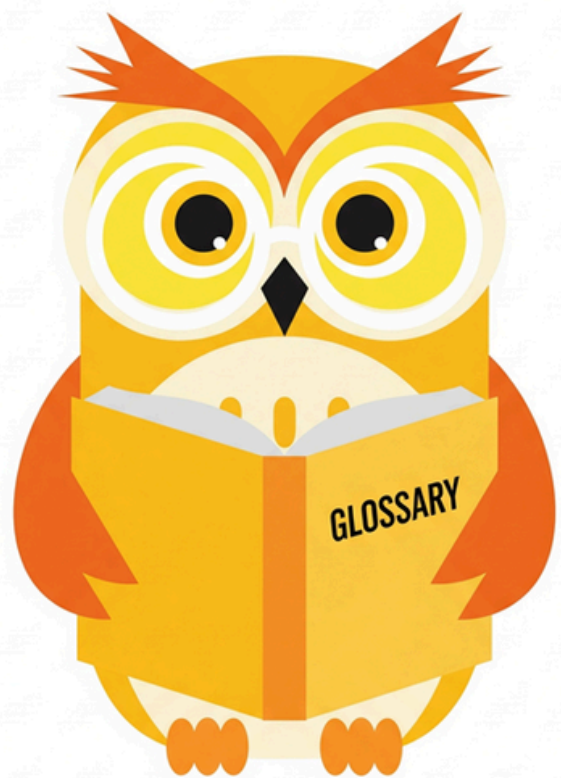
REFERENCIAS

- Imaginaformación. (s.f.). Cómo usar los arrays en C#. <https://imaginaformacion.com/tutoriales/como-usar-los-arrays-en-csharp>
- Lewis, J. (2023, 28 de abril). A quick tour of lists in C#. DEV Community. <https://dev.to/jlewis92/a-quick-tour-of-lists-in-c-2id5>
- Llamas, L. (s.f.). C# - Qué son diccionarios. Luis Llamas. <https://www.luisllamas.es/csharp-que-son-diccionarios/>
- Llamas, L. (s.f.). C# - Qué son tuplas. Luis Llamas. <https://www.luisllamas.es/csharp-que-son-tuplas/>
- Microsoft. (2024, 4 de julio). Lenguaje de expresiones regulares - Referencia rápida. Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/standard/base-types/regular-expression-language-quick-reference>
- NetMentor. (2021, 20 de octubre). Diccionarios en C#. <https://www.netmentor.es/entrada/diccionarios-csharp>
- NetMentor. (2021, 28 de julio). Tuple y ValueTuple en C#. <https://www.netmentor.es/entrada/tuple-valuetuple>
- Stackify. (2023, 19 de diciembre). C# Read File: A Beginner's Guide to Reading Text Files in C#. <https://stackify.com/c-read-file-a-beginners-guide-to-reading-text-files-in-c/>
- Universidad de Valladolid. (s.f.). Concepto de fichero. Fundamentos de Informática. https://www2.eii.uva.es/fund_inf/cpp/temas/10_ficheros/concepto_fichero.html



Ejemplo práctico

Crearás el ejemplo práctico que tu impartirás en la clase como refuerzo de la teoría dada.



Practiquemos lo aprendido

Crearás la actividad donde los alumnos practicarán por sí solos los conocimientos abordados en esta sesión.



Nombre de la actividad:

Cantidad de participantes:

Doy fe que esta actividad está
planificada en dtt (Sí/No):

Hora de inicio:

Hora de fin:

Duración (min):

Participantes: llenar las siguientes cajas de texto (tomar información del chat del meet)

Recursos

[LINK DEL VIDEO](#)



DUDAS ¿?



**¡GRACIAS POR SU
ATENCIÓN!**

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA